```
layer every 500 steps. 4. Run experiments across all datasets and both
complexity levels, tracking effective rank alongside existing metrics. 5.
Plot the evolution of effective rank alongside grokking curves for each
complexity level and operation. 6. Analyze how effective rank changes
before, during, and after grokking, and how this relates to task
complexity. 7. Investigate correlations between effective rank dynamics and
grokking speed or generalization performance. 8. Compare effective rank
patterns across different operations and model sizes. 9. Contrast effective
rank dynamics between operations that grok quickly versus those that grok
slowly or fail to grok. 10. Experiment with using effective rank as an
indicator for the onset of grokking.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 34/50 - lucky_initializations_grokking

```
"Name": "lucky_initializations_grokking",
"Title": "Lucky Initializations in Grokking: Identifying and Analyzing
Favorable Starting Points for Algorithmic Learning",
"Experiment": "1. Implement a function to generate and store 50 random
initializations for the Transformer model. 2. Modify the training loop to
support training from stored initializations and different learning rates.
3. For each dataset, train models from the 50 initializations with 3
learning rates, tracking 'grokking efficiency' (ratio of validation
accuracy to training steps at 99%
initializations (top 20%
characteristics of lucky initializations: weight distribution statistics,
layerwise norms, and attention pattern initialization. 6. Implement a
function to visualize the loss landscape around initial points using
filter-wise normalization. 7. Compare lucky initializations across
different operations to identify common patterns. 8. Develop a simple
predictor for initialization 'luckiness' based on identified
characteristics. 9. Test transfer of lucky initializations across tasks and
learning rates.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 35/50 - relative_attention_grokking

```
"Name": "relative_attention_grokking",
"Title": "Relative Positional Attention and Its Impact on Grokking in
Algorithmic Learning",
"Experiment": "1. Modify the DecoderBlock class to support two attention
types: standard (current) and relative positional. 2. Implement relative
positional attention, ensuring it works with the existing sequence length.
3. Update the Transformer class to accept an attention_type parameter. 4.
Run experiments for both attention types across all datasets, tracking:
time to grokking (99%
training loss, grokking transition sharpness (rate of validation accuracy
increase), and post-grokking stability (variance in validation accuracy
```

```
after reaching 99%
highlighting grokking points and transition periods. 6. Visualize and
compare attention patterns between the two mechanisms at key stages: pre-
grokking, during grokking transition, and post-grokking. 7. Analyze how
relative positional attention affects grokking behavior, transition
sharpness, and stability for each operation type compared to standard
attention. 8. Investigate correlations between attention type and grokking
speed or post-grokking stability. 9. Discuss implications for designing
transformers for specific algorithmic tasks based on findings.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Idea 36/50 - grokking_task_interference

```
"Name": "grokking_task_interference",
"Title": "Grokking and Task Interference: Exploring the Stability of
Algorithmic Understanding",
"Experiment": "1. Modify the training loop to support learning two modular
arithmetic operations sequentially (e.g., addition then multiplication). 2.
Implement a task scheduler that switches between tasks at regular
intervals. 3. Create a 'dual-task evaluation' function to assess
performance on both tasks simultaneously. 4. Track metrics: time to
grokking for each task, performance on the first task while learning the
second, and a 'grokking stability' score (maintenance of >95%
task 1 while learning task 2). 5. Run experiments with different task
switching frequencies. 6. Analyze how grokking on one task affects learning
speed and grokking on the subsequent task. 7. Visualize attention patterns
before and after introducing the second task to understand representation
changes. 8. Investigate the correlation between grokking speed on the first
task and stability of that understanding when learning the second task. 9.
Compare results with a baseline of learning both tasks simultaneously to
isolate the effects of sequential learning.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 37/50 - attention_inductive_bias_grokking

```
"Name": "attention_inductive_bias_grokking",
"Title": "Inductive Biases in Attention Mechanisms: Their Impact on
Grokking in Algorithmic Learning",
"Experiment": "1. Modify DecoderBlock class to support two attention
mechanisms: standard dot-product and additive (Bahdanau). 2. Implement
these attention mechanisms, ensuring compatibility with existing
architecture. 3. Update Transformer class to accept an attention_type
parameter. 4. Select a subset of most illustrative datasets based on
preliminary experiments. 5. Run experiments for each attention type on
selected datasets, tracking: time to grokking (99%
final validation accuracy, training loss, and 'grokking transition
sharpness' (defined as the maximum rate of validation accuracy increase
over any 500-step window). 6. Implement a simple generalization test using
```

```
slightly out-of-distribution examples (e.g., larger numbers for modular
arithmetic). 7. Plot learning curves for each attention type, highlighting
grokking points and transition periods. 8. Analyze how different attention
mechanisms affect grokking behavior, transition sharpness, and
generalization performance for each operation type. 9. Visualize attention
patterns for each mechanism at key stages: pre-grokking, during grokking
transition, and post-grokking. 10. Discuss implications for designing
transformers with appropriate inductive biases for specific types of
algorithmic learning tasks.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 38/50 - gradient_dynamics_grokking

```
"Name": "gradient_dynamics_grokking",
"Title": "Gradient Dynamics in Grokking: Analyzing Information Flow
Efficiency During Algorithmic Learning",
"Experiment": "1. Modify the training loop to compute gradient statistics
(sparsity and magnitude distribution) for each layer. 2. Implement
functions to calculate gradient sparsity (%
magnitude percentiles. 3. Update training process to store these metrics
every 500 steps. 4. Run experiments across all datasets, tracking gradient
metrics alongside existing performance metrics. 5. Plot the evolution of
gradient sparsity and magnitude distributions alongside grokking curves. 6.
Analyze how gradient dynamics change before, during, and after grokking. 7.
Compare gradient patterns between operations that grok quickly vs. slowly.
8. Investigate correlations between changes in gradient dynamics and
grokking speed or generalization performance. 9. Visualize gradient flow
patterns at key stages: pre-grokking, during grokking transition, and post-
grokking.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Idea 39/50 - adaptive_curriculum_grokking

```
"Name": "adaptive_curriculum_grokking",
"Title": "Adaptive Curriculum Learning in Grokking: Optimizing Example
Difficulty for Efficient Algorithmic Understanding",
"Experiment": "1. Modify AbstractDataset to include a difficulty scoring
function (e.g., input magnitude for modular arithmetic, cycle length for
permutations). 2. Implement adaptive sampling strategy: start with easiest
20%
level exceeds 90%
tracking difficulty of selected examples. 4. Run experiments comparing
adaptive curriculum, random sampling, and static curriculum (increasing
difficulty linearly) across all datasets. 5. Track metrics: time to
grokking, final validation accuracy, learning trajectory smoothness, and
example difficulty distribution over time. 6. Analyze relationship between
difficulty progression and grokking onset. 7. Visualize learning curves and
difficulty progression for each strategy. 8. Compare consistency and speed
```